

## **1. Systems decision that I changed my mind on**

I originally ran RAG evaluation manually before releases because I thought automated checks would slow us down. After a bad retrieval change slipped through and burned a week of firefighting, I flipped: RAGAS gates in CI on every retrieval change. Slower per commit, way faster per month. I now default to automating any check I've done manually twice.

## **2. Sidequest outside core responsibilities**

The marketing team at my org ran reporting off 3 to 5 days of manual Excel across eight data sources. Nobody asked me to fix it. I built a Medallion pipeline on Databricks with a data quality engine and auto-refreshed tables on my own time because watching people copy-paste CSVs was physically painful. Reporting lag went from days to same-day.

## **3. Changed how I use LLMs**

I used to fine-tune for every task. QLoRA made it cheap, so it was my hammer. Then I noticed our RAG pipeline with good chunking beat a fine-tuned model on contract QA while being fixable in hours, not training runs. Now I exhaust retrieval and prompting first and fine-tune only when the task needs behavior the base model can't reach. Fewer GPUs, faster iteration.

## **4. How I am a systems person**

Telecom taught me this: I worked where one escaped defect meant SLA penalties across 100+ carrier networks. My reflex since is boring and consistent. Anything done twice gets automated, every pipeline logs its own failures, and tests gate merges (202 pytest tests on my last personal project, nobody made me). Structure is what lets a small team move fast without babysitting things.